Project Name: Chula Fondue

Group Members:

a. Kotchapan Lertpanyaroj 6630001321
b. Chavapon Suwanna 6630055221
c. Natamathee Polerdsawat 6630092421
d. Piyapanchanok Meeboonsalang 6630196021
e. Pitchanan Thaiphayak 6632151121

Problem Description:

Chulalongkorn University is a large campus with dozens of buildings, faculties, and public spaces. Every day, students, staff, and visitors run into common maintenance problems like broken air conditioners, damaged sidewalks, overflowing trash cans, or burnt-out lights.

Without a single place to report these problems, people face three main issues:

- Unclear Channels: People rarely know which specific office or department is in charge of fixing a problem in a certain area.
- Lost Reports: Complaints sent through word of mouth, social media, or paper forms are easily lost or forgotten.
- No Updates: People cannot see if anyone is actually fixing their issue, leading to frustration and repeated reports for the same problem.

Chula Fondue solves this by providing one central website where anyone can report a campus issue, central admins can assign the task to the right repair team, and users can track the progress every step of the way.

Target Customer:

Central Admin: Full system access, reviews incoming issue reports, routes tickets to appropriate faculty/department queues, maintains the department list, monitors overall response metrics.

Department Staff (Resolver): Views assigned tickets for their specific faculty or department, updates work status, uploads completion photos, resolves tickets.

Reporter (Student/Staff/Visitor): Submits campus issues with photos and locations, views ticket progress, receives notifications, closes tickets upon verification.

Scenario (use-case & description):

Use Case Name: Report Campus Issue	**ID:** UC-01 (Main use-case)	**Important Level:** High
Primary Actor: Reporter (Student / Staff / Visitor)		**Use Case Type:** Business / Management
Stakeholders and Interests: - Reporter: wants to report a campus issue quickly without needing to know which department manages it. - Admin: wants accurate issue details and location to route the ticket correctly.		
Brief Description: Reporter submits a campus issue report including category, description, location, and up to 5 photos (maximum 5 MB each). Photos are uploaded to Cloudinary, and their URLs are stored with the ticket.		
Trigger: User encounters a maintenance or infrastructure issue on campus grounds. **Type:** External		
Relationships: None.		
Precondition: User is authenticated or logged in as guest.		
Postcondition: Ticket created with status "Submitted" and queued for Central Admin review.		
Normal Flow of Events: 1. Reporter accesses Chula Fondue web portal and selects "Report Issue". 2. Reporter inputs issue category, location (Faculty/Building), and detailed description. Guest reporters also provide a contact email. 3. Reporter uploads 1–5 photos of the issue. 4. The API Gateway validates and uploads photos to Cloudinary. 5. Ticket Service validates the report, generates a unique Ticket ID, stores the ticket and photo URLs in PostgreSQL, and sets its status to "Submitted". 6. The ticket appears in the Central Admin inbox.		
Subflows: None.		
Alternate/Exceptional Flow: - Required information is missing or the photo is invalid; the system shows an error message and the ticket is not created. - Reporter cancels the ticket while its status is "Submitted"; ticket status becomes "Cancelled" and the ticket is removed from the Central Admin inbox.		

Use Case Name: Route Ticket to Department	**ID:** UC-02 (Main use-case)	**Important Level:** High
Primary Actor: Central Admin		**Use Case Type:** Business / Management

Stakeholders and Interests:
- Admin: wants to review and dispatch tickets accurately to avoid misrouting.
- Department Staff: wants to receive clear, pre-screened tickets relevant to their jurisdiction.

Brief Description:
Central Admin reviews a submitted issue report, verifies details, and dispatches the ticket to the responsible department queue.

Trigger: New ticket submitted to the central queue.
Type: External

Relationships: Requires an existing submitted ticket from UC-01 and uses department information maintained through UC-08. A successful assignment triggers asynchronous notification processing (UC-05).

Precondition: Ticket status is "Submitted"; Admin authenticated.

Postcondition: Ticket status updated to "Assigned" and transferred to the department queue.

Normal Flow of Events:
1. Admin logs in and opens the central admin dashboard.
2. Admin reviews incoming ticket details, photo, and reported location.
3. Admin selects the responsible department from the department list.
4. Admin submits assignment.
5. The Ticket Service validates the selected department through the Department Service, updates the ticket status to "Assigned" in PostgreSQL, and publishes a TicketAssigned event to RabbitMQ. The Notification Service consumes the event and stores the notification record in MongoDB.

Subflows:
- Admin rejects invalid, duplicate, or spam tickets with a written reason; ticket status becomes "Rejected" and the reporter is notified.
- Admin re-routes an assigned ticket by assigning it to another department.

Alternate/Exceptional Flow:
The selected department no longer exists; the system shows an error and asks the Admin to select again.

Use Case Name: Resolve Ticket	**ID:** UC-03 (Main use-case)	**Important Level:** High
Primary Actor: Department Staff		**Use Case Type:** Operational / Maintenance

Stakeholders and Interests:
- Department Staff: wants to track assigned tasks and record work completion.
- Reporter: wants verification that the issue has been resolved.

Brief Description:
Department staff inspects the reported issue, executes repairs, uploads completion photo proof, and marks ticket as resolved.

Trigger: Ticket assigned to department queue.
Type: External

Relationships: Includes UC-05 for status-change notification.

Precondition: Ticket status is "Assigned" or "In Progress".

Postcondition: Ticket marked as "Resolved" with completion proof attached.

Normal Flow of Events:
1. Department Staff views tickets assigned to their department.
2. Staff changes the ticket status to "In Progress".
3. Ticket Service updates the ticket status in PostgreSQL and publishes a status-change event to RabbitMQ.
4. Staff performs the required maintenance.
5. Staff uploads completion photos through the API Gateway to Cloudinary.
6. Staff marks the ticket as "Resolved".
7. Ticket Service stores the completion photo URLs, updates the ticket status, and publishes a TicketResolved event to RabbitMQ.

Subflows: None.

Alternate/Exceptional Flow: None.

Use Case Name: View Ticket Status & History	**ID:** UC-04	**Important Level:** Medium
Primary Actor: Reporter		**Use Case Type:** User Service / Informational

Stakeholders and Interests:
- Reporter: wants real-time transparency on submitted ticket progress.

Brief Description:
Reporter checks current status and resolution details of their submitted tickets.

Trigger: User visits platform to check update.
Type: External

Relationships: None.

Precondition: Ticket exists in system.

Postcondition: Status details displayed to the user.

Normal Flow of Events:
1. The reporter opens the "My Tickets" section.
2. The system displays active and historical ticket list with current status indicators.
3. Reporter clicks a ticket to view progress timeline, admin updates, and completion photos.

Subflows: None.

Alternate/Exceptional Flow:
- Ticket ID not found; system displays error message.

Use Case Name: Record Ticket Notification	**ID:** UC-05	**Important Level:** Medium
Primary Actor: None **Supporting Actor:** RabbitMQ (Messaging Infrastructure)		**Use Case Type:** Automated / Integration

Stakeholders and Interests:

- Reporter: wants real-time transparency on submitted ticket progress.

Brief Description:
The system asynchronously processes ticket status-change events using RabbitMQ. The Notification Service consumes events and stores notification records in MongoDB. Email delivery through Gmail API is planned for a future iteration.

Trigger: Ticket status transition occurs (assigned, in progress, resolved, rejected, or reopened).
Type: System-triggered

Relationships: Relies on Gmail API; included by UC-02, UC-03, and UC-06.

Precondition: Ticket status has changed; the recipient's email address is available.

Postcondition: Email request sent to Gmail API.

Normal Flow of Events:

1. A ticket status change occurs in Ticket Service.
2. Ticket Service publishes a notification event to RabbitMQ.
3. RabbitMQ routes the event to the notification queue.
4. Notification Service consumes the event.
5. Notification Service stores the notification record in MongoDB.
6. Notification Service acknowledges the message after successful processing.

Subflows: None.

Alternate/Exceptional Flow:

- If RabbitMQ is temporarily unavailable, Ticket Service logs the publishing failure without rolling back the completed ticket status update.
- If Notification Service fails before acknowledging a message, RabbitMQ may redeliver it.
- Notification Service uses a unique event ID to prevent duplicate notification records.

Use Case Name: Confirm or Reopen Ticket Resolution	**ID:** UC-06	**Important Level:** Medium
Primary Actor: Reporter (Student / Staff / Visitor)		**Use Case Type:** User Verification / Workflow Transition

Stakeholders and Interests:

- Reporter: wants to verify that the reported issue has actually been fixed before the ticket is permanently closed.
- Department Staff: wants clear feedback on whether the repair met the reporter's satisfaction.

Brief Description:
Reporter reviews a ticket marked as "Resolved", then either confirms the fix to close the ticket or reopens it with a reason if the problem persists.

Trigger: Ticket status transitions to "Resolved".
Type: External / User-triggered

Relationships: Follows UC-03; includes UC-05 for status-change notification.

Precondition: Ticket status is "Resolved".

Postcondition: Ticket status updated to "Closed" or reverted to "In Progress".

Normal Flow of Events:

1. Reporter checks the ticket status through the Chula Fondue web portal and sees that the ticket has been marked as "Resolved".
2. Reporter opens the ticket details and reviews the completion photo proof submitted by staff.
3. Reporter clicks "Confirm Resolution".
4. System calls ConfirmResolution(), updates ticket status from "Resolved" to "Closed", and archives the ticket.

Subflows: None.

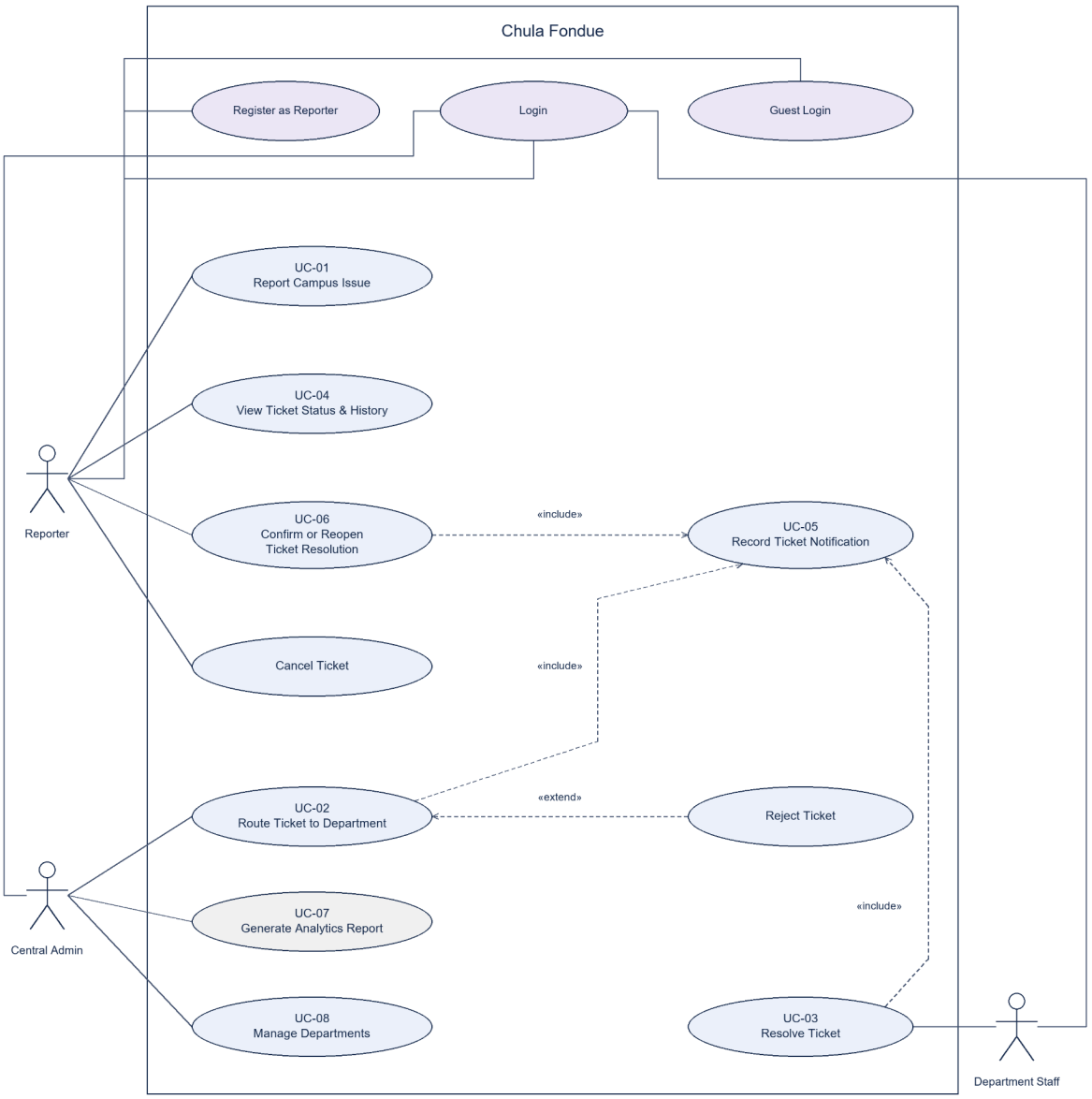
Alternate/Exceptional Flow:

- When a reporter reopens a resolved ticket, Ticket Service updates the status to "In Progress" and publishes a TicketReopened event to RabbitMQ for asynchronous notification processing.
- Auto-Closure: If the reporter does not take action within 7 days, the system automatically transitions the status from "Resolved" to "Closed" (planned for a later iteration).

Use Case Name: Generate Analytics Report (Planned for Future Iteration)	**ID:** UC-07	**Important Level:** Medium
Primary Actor: Central Admin		**Use Case Type**: Business Intelligence / Analytical
Stakeholders and Interests: - Admin / Executive: wants visual data insights regarding average resolution times and high-frequency problem areas across faculties.		
Brief Description: This feature is part of the overall system design but is not included in the Progress #2 implementation scope.		
Trigger: Admin navigates to the Analytics section. **Type:** Internal / User-triggered		
Relationships: Reads aggregate data generated from UC-01, UC-02, and UC-03.		
Precondition: Ticket data exists in the system; Admin authenticated.		
Postcondition: Visual dashboard and statistical tables rendered to Admin.		
Normal Flow of Events: 1. Admin selects date ranges, specific faculties, or issue categories. 2. The system retrieves the required ticket data and aggregates it according to the selected filters. 3. System renders visual charts showing Mean Time to Resolve (MTTR) and recurring issue frequencies per department/location.		
Subflows: None.		
Alternate/Exceptional Flow: - No data found for selected filters; system displays an empty state notification.		

Use Case Name: Manage Departments	**ID:** UC-08	**Important Level:** Medium
Primary Actor: Central Admin		**Use Case Type:** Business / Management
Stakeholders and Interests: Central Admin wants an accurate, up-to-date list of departments so that tickets can be routed and notified correctly.		
Brief Description: Central Admin maintains the list of departments that tickets can be routed to.		
Trigger: A department is added, changed, or no longer handles issues. **Type:** External		
Relationships: Provides the department list used in UC-02.		
Precondition: Admin authenticated.		
Postcondition: Department list is updated.		
Normal Flow of Events: 1. Admin opens the department management page. 2. System displays the current department list. 3. Admin creates a new department, or edits or deletes an existing one (name, contact email, buildings covered). 4. System validates the input and saves the change.		
Subflows: None.		
Alternate / Exceptional Flow: Department name already exists; system rejects the change with an error message.		

Use Case Diagram



Functional Requirements

User authentication and profile

- The system shall authenticate registered users using JWT-based authentication.
- The system shall allow users to access the platform using guest credentials.
- The system shall enforce role-based access control (RBAC) for Reporter, Central Admin, and Department Staff.

Issue reporting and media upload

- The system shall allow users to create a new issue report.
- The system shall allow users to upload up to five photos per ticket, with a maximum size of 5 MB per photo, using Cloudinary.
- The system shall allow users to specify the issue location by building or faculty.
- The system shall allow users to select an issue category.
- The system shall allow users to provide a detailed text description of the issue.

Central admin ticket orchestration

- The system shall allow central administrators to review incoming tickets through a centralized dashboard.
- The system shall allow central administrators to categorize incoming tickets.
- The system shall allow central administrators to manually assign tickets to designated department queues.
- The system shall allow central administrators to re-route tickets to other department queues.
- The system shall allow Central Admin to reject a ticket with a written reason using RejectTicket(); its status becomes Rejected and the reporter is notified.

Department management

- The system shall allow Central Admin to create, view, update, and delete departments (name, contact email, buildings covered).

Ticket lifecycle and status tracking

- The system shall maintain the following ticket statuses: Submitted, Assigned, In Progress, Resolved, Closed, Rejected, Cancelled.
- The system shall track each ticket as it transitions from one status to another.
- The system shall allow Department Staff to upload completion photo proof when resolving a ticket.

Departmental queue management

- The system shall allow departmental staff to view tickets assigned to their unit.
- The system shall allow departmental staff to filter tickets assigned to their unit.
- The system shall allow departmental staff to update tickets assigned to their unit.

Notification system

- The system shall asynchronously process ticket status-change events through RabbitMQ.
- The Notification Service shall store notification records in MongoDB.
- The system shall notify the assigned department when a ticket is assigned or reopened.
- Email delivery through Gmail API is planned for a future iteration.

Ticket Cancellation

- The system shall allow reporters to cancel a ticket while its status is Submitted using CancelTicket(); its status becomes Cancelled and it is removed from the Central Admin inbox.

Ticket Closure & Feedback System

- The system shall allow reporters to confirm ticket resolution and update ticket status from Resolved to Closed.
- The system shall allow reporters to reopen a Resolved ticket by providing a mandatory reason, reverting status back to In Progress.
- The system shall automatically transition Resolved tickets to Closed if no action is taken by the reporter within 7 days (planned for a later iteration).

Analytics & Visual Reporting System

- The system shall provide a dashboard visualizing average resolution time (Mean Time to Resolve) filtered by issue category and department.
- The system shall identify and display recurring issue hotspots by location and faculty.

Non-Functional Requirements

Usability

- The system shall provide a fully responsive interface, optimizing the reporting flow for mobile browsers without requiring native app installation.
- The system shall provide a primarily Thai-language user interface, while commonly used technical terms may remain in English.

Performance

- The system shall load public reporting interfaces in under 2 seconds.
- The system shall process image uploads within 3 seconds under normal load.

- The system shall respond to ticket status/history queries within 1 second.

Availability

- The system shall maintain 99.5% operational uptime during active academic terms.

Security

- The system shall enforce role-based access control (RBAC) with strict boundary permissions between Reporters, Central Admins, and Department Staff.
- The system shall enforce authenticated sessions for all non-guest actions.

Scalability

- The system shall accommodate high concurrent traffic spikes (up to 1,000 active users) without degrading ticket submission performance.

Operational

- The system shall display properly on both desktop and mobile devices, and support major browsers and OS (Windows, macOS, iOS, Android).

Architecture Decision Records (ADRs):

ADR-001: Frontend Framework

Context

Chula Fondue needs a responsive interface for Reporter, Department Staff, and Central Admin, with Thai-language screens, photo upload, and ticket tracking. The team considered three options:

- **React**

  Pros: Reusable components and TypeScript support fit the shared forms, ticket cards, and dashboards.

  Cons: The team must select and configure routing, state management, and styling conventions.

- **Vue**

  Pros: Its progressive approach and single-file components support incremental development and readable templates.

  Cons: The team would need to learn Vue-specific reactivity and component conventions and agree on supporting tools.

- **Angular**

  Pros: Built-in routing and dependency injection provide consistent application structure.

  Cons: Its broader framework concepts add learning and setup work for this project's timeline.

Decision

We will use React + TypeScript for the frontend.

Status

Accepted

Consequences

The team will build reusable components for reporting forms, ticket cards, status badges, and role-specific dashboards.

The team must learn React component patterns, Hooks, and TypeScript.

Routing, state management, and styling conventions must be selected and maintained.

Shared TypeScript types and API contracts must stay aligned with the backend.

ADR-002: Backend Framework

Context

The backend must support role-based access, image uploads, department management, ticket workflows, analytics, and Gmail API integration.Backend services use synchronous REST/gRPC communication for request-response operations and asynchronous RabbitMQ messaging for ticket notification events. Notification processing failures must not roll back successfully completed ticket status changes.. Automatic notification retry is outside this iteration.

- **Go**

  Pros: Built-in concurrency primitives and compiled executables suit independent services.

  Cons: The team's limited Go experience adds learning work, and a second language requires separate contracts and conventions.

- **Node.js + Express + TypeScript**

  Pros: A minimal HTTP framework offers flexibility and allows TypeScript across the frontend and backend.

  Cons: The team must establish service structure, dependency injection, validation, and testing conventions.

- **Node.js + NestJS + TypeScript**

  Pros: Modules, dependency injection, guards, and gRPC support provide a consistent structure for separate services.

Cons: Decorators, dependency injection, and microservice configuration introduce a learning curve.

Decision

We will use Node.js with NestJS (TypeScript) for all backend services. Each microservice is a separate NestJS application.

Status

Accepted

Consequences

The team must allocate time to learn NestJS modules, decorators, dependency injection, guards, and service configuration.

TypeScript will be used across the frontend and backend.

The team must maintain shared conventions and contracts across separate NestJS applications, including DTOs, status definitions, REST conventions, and .proto contracts.

Each service requires its own configuration, deployment, and operational monitoring.

RabbitMQ is used to decouple Ticket Service from Notification Service. The team must manage message acknowledgment, redelivery, idempotency, and publishing failures.

ADR-003: Central Admin Orchestration vs. Automated AI Classification

Context

We initially considered using an AI classification model (such as an LLM or text classifier) to automatically parse issue descriptions and route tickets directly to responsible departments. However, campus administrative structures involve overlapping jurisdictions (e.g., central grounds vs. specific faculty maintenance units). Misclassified AI assignments would cause ticket bouncing between departments, delaying repairs. Manual orchestration by central administrators ensures higher routing accuracy, provides human oversight, and filters out spam or duplicate reports before reaching maintenance teams.

Decision

Implement a manual central admin orchestration workflow where human administrators review incoming tickets and dispatch them to the correct department queues.

Status

Accepted

Consequences

- Ensures precise ticket assignment and eliminates AI misrouting errors.

- Prevents spam and duplicate tickets from reaching handling departments.
- Introduces dependency on central admin availability, slightly increasing dispatch latency.

ADR-004: Notification Processing Strategy (Synchronous vs. Asynchronous Messaging)

Context
Initially, we considered implementing only an in-app notification system where updates appear strictly when users log in to the Chula Fondue portal. However, this approach requires users to constantly check the site manually, resulting in low engagement and delayed awareness of issue resolutions. Since Chulalongkorn University operates on the Google Workspace ecosystem, all students and staff already utilize Gmail. We evaluated integrating directly with the **Gmail API** via OAuth2 to deliver automated, real-time notifications straight to users' email inboxes instead of relying solely on in-app updates or managing internal mail infrastructure.

Decision
We will use RabbitMQ for asynchronous ticket notification processing. Ticket Service publishes notification events, while Notification Service consumes and stores them in MongoDB. Gmail API integration is deferred to a future iteration.

Status
Accepted

Consequences

- Ticket Service does not need to wait for Notification Service to process notification events.
- Notification Service can process queued events independently.
- RabbitMQ introduces additional infrastructure and operational complexity.
- Duplicate delivery must be handled through idempotent processing.
- Event publishing failures may result in lost notifications unless a reliable publishing mechanism is introduced.

ADR-005: Service Communication Style (REST vs. gRPC)

Context

The browser needs an HTTPS interface, while backend services need clear contracts for ticket workflows, analytics, notifications, and department lookup. REST provides HTTP resource conventions; gRPC provides .proto service contracts. The current iteration uses synchronous calls and must complete ticket status updates even when notification delivery fails.

Decision

- Client → API Gateway: REST/HTTPS
- API Gateway → Auth Service: REST
- API Gateway → Ticket Service: gRPC
- API Gateway → Department Service: REST
- Ticket Service → Department Service: REST
- Ticket Service → RabbitMQ: AMQP
- Notification Service consumes messages from RabbitMQ.
- Notification Service → MongoDB: MongoDB Driver
- API Gateway → Cloudinary: HTTPS
- Analytics Service is planned for a future iteration.

The architecture combines synchronous REST/gRPC communication with asynchronous RabbitMQ messaging. Ticket operations are completed independently of notification processing.

Ticket Service requests a notification as part of its status-change response; API Gateway invokes Notification Service via gRPC, and Notification Service calls Gmail API via HTTPS.

Status

Accepted

Consequences

- The API Gateway translates REST requests into gRPC calls for Ticket Service.
- RabbitMQ decouples ticket operations from notification processing.
- Notification Service must handle message acknowledgment and duplicate delivery.
- Each service maintains ownership of its own database.
- RabbitMQ failures must be logged and handled without corrupting ticket data.

Architecture Diagram

The diagram below shows the synchronous communication paths defined in ADR-005. Department Service supports UC-08 by maintaining department names, contact emails, and buildings covered; UC-02 uses this list to validate routing.

ADR-006: Message Broker Selection

Context

The system requires asynchronous communication between Ticket Service and Notification Service. Direct synchronous calls introduce coupling and make notification processing dependent on service availability.

Decision

We will use RabbitMQ as the message broker for ticket notification events.

Status: Accepted

Consequences

RabbitMQ enables asynchronous processing and message redelivery but introduces operational complexity. Consumers must acknowledge messages after successful processing and handle duplicate deliveries.

ADR-007: Database Selection

Context

Different services have different data storage requirements. Ticket, Department, and Authentication services require relational data, while Notification Service stores event-based notification records.

Decision

We will use PostgreSQL hosted on Render for Auth Service, Ticket Service, and Department Service. MongoDB will be used for Notification Service.

Status: Accepted

Consequences

Each service owns its database and must not directly access another service's data. Database migrations, connection management, backups, and availability must be managed independently.

ADR-008: Authentication Strategy

Context

The system must support registered users, guest reporters, and role-based permissions for administrators and department staff.

Decision

We will use JWT-based authentication with role-based access control (RBAC). Guest reporters will be supported through a restricted guest access mechanism.

Status: Accepted

Consequences

The system must validate JWT signatures and expiration, enforce role permissions, and prevent guest users from accessing protected administrative operations.